Lista de exercícios – Tratamento de Exceções

Orientações para resolução da lista

- Esta lista trabalha exclusivamente o conteúdo de **tratamento de exceções**: a diferença entre erro de sintaxe e exceção, os blocos `try`, `except`, `else` e `finally`, o tratamento de exceções específicas (`ValueError`, `ZeroDivisionError`, `IndexError`, `FileNotFoundError`, `TypeError`), o uso de `except ... as erro`, a validação de entradas com repetição e o lançamento de exceções com `raise`.
- Todas as questões são de **construção de código**. Antes de implementar, liste quais exceções podem acontecer em cada trecho do programa e com quais entradas.
- Sempre que possível, trate **exceções específicas**, com mensagens claras que digam ao usuário o que ele deve corrigir. Evite o `except` genérico.
- Coloque dentro do `try` apenas o trecho que pode falhar. Use o `else` para o que só deve executar quando tudo deu certo e o `finally` para o que deve executar sempre.
- Lembre-se da divisão de responsabilidades vista em aula: a função ou o método **detecta** o problema e lança a exceção com `raise`; quem chamou **decide** o que fazer.
- Teste cada programa com entradas válidas e também com entradas que provoquem cada uma das exceções tratadas. O programa nunca deve ser interrompido com uma mensagem de *Traceback*.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

Exercícios

1. Faça um programa que leia dois números reais e exiba o resultado da divisão do primeiro pelo segundo, com duas casas decimais.

O programa deve tratar separadamente os dois problemas que podem acontecer, exibindo uma mensagem própria para cada um. Em qualquer situação, deve exibir ao final a mensagem `Calculadora encerrada.`

Exemplos de execução:

```
Dividendo: 7
Divisor: 2
Resultado: 3.50
Calculadora encerrada.
```

```
Dividendo: sete
Digite apenas numeros.
Calculadora encerrada.
```

```
Dividendo: 7
Divisor: 0
Nao e possivel dividir por zero.
Calculadora encerrada.
```

Utilize `else` para exibir o resultado e `finally` para a mensagem final.

2. Crie uma função chamada `ler_inteiro(mensagem, minimo, maximo)` que exiba a mensagem, leia um valor e só o devolva quando ele for um número inteiro dentro do intervalo de `minimo` a `maximo`, incluindo os extremos.

Enquanto o valor for inválido, a função deve repetir a leitura. Quando o texto não puder ser convertido, deve exibir a mensagem `Digite um numero inteiro.` e, quando o número estiver fora do intervalo, a mensagem `Valor fora do intervalo` seguida dos limites, como no exemplo abaixo.

Exemplo de execução para `ler_inteiro("Idade: ", 0, 120)`:

```
Idade: vinte
Digite um numero inteiro.
Idade: 150
Valor fora do intervalo [0, 120].
Idade: 20
```

No programa principal, use a função para ler o dia, o mês e o ano de nascimento de uma pessoa (dia entre 1 e 31, mês entre 1 e 12, ano entre 1900 e 2026) e exiba a data no formato `dd/mm/aaaa`.

3. Um restaurante guarda seu cardápio em uma lista:

```
cardapio = ["Sanduiche", "Suco", "Pao de queijo", "Cafe"]
precos = [15.0, 8.0, 4.0, 5.5]
```

Faça um programa que exiba as opções numeradas a partir de 1 e leia pedidos até que o cliente digite `0`. Para cada pedido, o programa deve ler o número da opção e a quantidade e acumular o valor da conta.

Trate os seguintes problemas, cada um com sua própria mensagem, sem interromper o programa:

- texto no lugar de um número (`ValueError`);
- opção que não existe no cardápio (`IndexError`).

Atenção: como visto em aula, com índices negativos o Python não lança `IndexError`: `cardapio[-1]` devolve o último item. Por isso, opções menores que 1 também devem ser tratadas como inexistentes. Uma forma de fazer isso é lançar você mesmo a exceção com `raise IndexError`.

Ao final, exiba o valor total da conta e a quantidade de pedidos inválidos.

4. Crie uma função chamada `calcular_media(notas)` que receba uma lista de notas e devolva a média. A função deve lançar um `ValueError` com a mensagem `"A lista de notas esta vazia."` quando a lista não tiver elementos e um `ValueError` com a mensagem `"Nota invalida: X"` quando alguma nota estiver fora do intervalo de 0 a 100, sendo X a nota inválida.

No programa principal, chame a função para as listas abaixo, tratando a exceção com `except ValueError as erro` e exibindo a mensagem recebida:

```
[80, 90, 70]
[]
[50, 120, 60]
```

A saída esperada é:

```
Media: 80.0
Erro: A lista de notas esta vazia.
Erro: Nota invalida: 120
```

Observe que a mensagem é definida dentro da função, mas é o programa principal que decide como exibi-la.

5. O arquivo `cardapio.txt` contém uma linha por produto, no formato `nome;preco`:

```
Sanduiche;15.00
Suco;oito
Pao de queijo;4.00
Cafe
Bolo;9.50
```

Faça um programa que leia o arquivo e monte um dicionário com os produtos e seus preços. O programa deve:

- exibir `Cardapio nao encontrado.` caso o arquivo não exista;
- avisar sobre cada linha com preço inválido, informando o número da linha, e ignorá-la;
- avisar sobre cada linha sem o separador `;`, também a ignorando;
- exibir `Fim da leitura do cardapio.` em qualquer situação.

Para o exemplo acima, o programa deve avisar sobre as linhas 2 e 4 e exibir os três produtos válidos com seus preços e o preço médio.

Dica: ao separar a linha `Cafe` com `split(";")`, a lista obtida tem um único elemento. Acessar a posição 1 dessa lista gera qual exceção?

6. Crie uma classe `ContaBancaria` com os atributos `titular` e `saldo` e os métodos `depositar(valor)` e `sacar(valor)`. Em vez de apenas imprimir avisos, os métodos devem **lançar exceções**:

- `depositar` deve lançar `ValueError("O valor do deposito deve ser positivo.")` quando o valor for menor ou igual a zero;
- `sacar` deve lançar `ValueError("O valor do saque deve ser positivo.")` quando o valor for menor ou igual a zero e `ValueError("Saldo insuficiente.")` quando o valor for maior que o saldo.

Em todos os casos, o saldo só pode ser alterado se a operação for válida.

No programa principal, leia uma sequência de operações no formato `tipo valor` (por exemplo, `d 100` para depósito e `s 30` para saque) até que seja digitado `fim`. Para cada operação, trate as exceções lançadas pelos métodos e também os erros de digitação (valor que não é número ou linha sem o valor), exibindo uma mensagem clara. Ao final, exiba o saldo e a quantidade de operações recusadas.

7. Crie uma função chamada `converter_tempo(texto)` que receba uma duração no formato `"mm:ss"` e devolva o total de segundos.

A função deve lançar `ValueError` com uma mensagem adequada quando:

- o texto não tiver exatamente uma vez o caractere `:`;
- os minutos ou os segundos não forem números inteiros;
- os segundos não estiverem entre 0 e 59 ou os minutos forem negativos.

Por exemplo:

```
converter_tempo("3:25")  -> 205
converter_tempo("0:59")  -> 59
converter_tempo("3:75")  -> ValueError: Segundos devem estar entre 0 e 59.
converter_tempo("3m25")  -> ValueError: Use o formato mm:ss.
converter_tempo("a:10")  -> ValueError: Minutos e segundos devem ser inteiros.
```

No programa principal, leia a duração de várias músicas até que seja digitado `fim`. Durações inválidas devem ser recusadas com a mensagem da exceção. Ao final, exiba a duração total do álbum no formato `mm:ss` e a música mais longa.

8. O arquivo `vendas.txt` contém uma linha por venda, no formato `produto,quantidade,preco`. Algumas linhas foram digitadas com erro.

Faça um programa que:

a) leia o nome do arquivo pelo teclado e, caso ele não exista, peça um novo nome, até que um arquivo existente seja informado;

b) processe todas as linhas, acumulando em um dicionário o valor total vendido por produto;

c) ignore as linhas com problemas, gravando cada uma delas no arquivo `erros.txt`, acompanhada do número da linha e do motivo (campos faltando, quantidade inválida ou preço inválido);

d) exiba o total por produto, o total geral e a quantidade de linhas ignoradas.

Exemplo de conteúdo de `vendas.txt`:

```
Caneta,10,2.5
Caderno,dois,18.0
Caneta,4,2.5
Lapis,20
Caderno,1,18.0
```

Para esse exemplo, o arquivo `erros.txt` deve conter:

```
Linha 2: quantidade invalida
Linha 4: campos faltando
```

Para identificar o motivo, use um `try` separado para cada conversão.

9. Crie as classes `Produto` (nome e preço), `ItemPedido` (**tem um** produto e uma quantidade) e `Pedido` (**tem vários** itens).

A classe `Pedido` deve ter os métodos:

- `adicionar_produto(produto, quantidade)`: lança um `ValueError` quando a quantidade for menor ou igual a zero e outro quando a soma da quantidade já pedida daquele produto com a nova quantidade ultrapassar 10, com as mensagens abaixo. Só cria e guarda o item se tudo estiver válido;

```
A quantidade deve ser maior que zero.
Limite de 10 unidades por produto.
```

- `remover_produto(nome)`: lança `ValueError` quando o produto não estiver no pedido;
- `exibir_resumo()`: mostra os itens e o total.

No programa principal, monte um menu com as opções *adicionar*, *remover*, *resumo* e *sair*. As opções, os números dos produtos de um cardápio e as quantidades devem ser lidos com uma função `ler_inteiro` que repete a leitura até receber um número inteiro. Trate as opções inexistentes do cardápio e todas as exceções lançadas pelo pedido, exibindo `Erro:` seguido da mensagem.

10. Uma escola deseja cadastrar as notas de uma turma. Faça um programa que:

a) leia o nome do arquivo de saída e o número de estudantes da turma (um inteiro positivo, lido com validação);

b) para cada estudante, leia o nome e a nota com a função `ler_nota()`. A nota deve estar entre 0 e 100: se o usuário digitar texto, a função deve pedir novamente; se a nota estiver fora do intervalo, a função deve informar o problema e pedir novamente. Nomes vazios também devem ser recusados;

c) ao final, grave no arquivo informado uma linha por estudante, no formato `nome;nota`, e exiba a média da turma, a maior nota e o nome de quem a obteve.

A função `ler_nota()` deve ser construída a partir da função auxiliar `validar_nota(texto)`, que converte o texto e lança um `ValueError` quando a nota estiver fora do intervalo, com a mensagem *A nota deve estar entre 0 e 100.* Observe que a conversão com `float()` também lança um `ValueError`, porém com outra mensagem: use mensagens diferentes para os dois casos.

Em seguida, acrescente ao programa uma segunda etapa que leia de volta o arquivo gravado e exiba a quantidade de aprovados (nota maior ou igual a 60). Essa etapa deve tratar o caso de o arquivo não existir e ignorar, com aviso, linhas que estejam fora do formato. Utilize `finally` para exibir `Leitura encerrada.` ao terminar essa etapa.